

CAN BASED SMART VEHICAL SUB-SYSTEM SIMULATION & DIAGNOSTICS

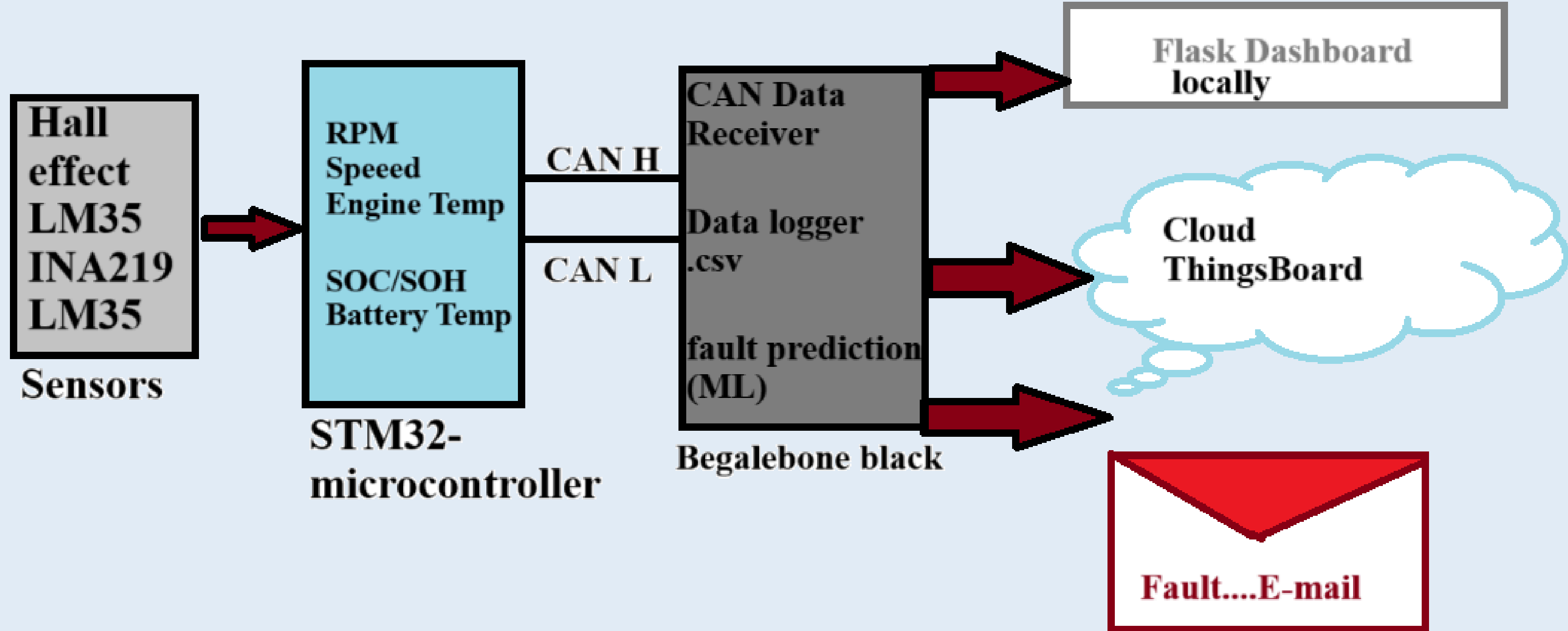
Guided by: Shripad Deshpande



PROJECT OVERVIEW

- Simulate a vehicle sub-system using CAN protocol for real-time diagnostics.
- Integrate IoT and ML for predictive fault detection.
- Key Features:
- Multi-node CAN communication (STM32 + BeagleBone).
- Cloud (ThingsBoard) and local (Flask) dashboards.
- Automated email alerts for faults.

SYSTEM ARCHITECTURE



Hardware Components

Microcontrollers/Boards

- STM32F407VGT6 (Node-1 and Node-2)
- BeagleBone Black as gateway (Central Node-3).

CAN Transceivers:

- MCP-2551
- SN65HVD230

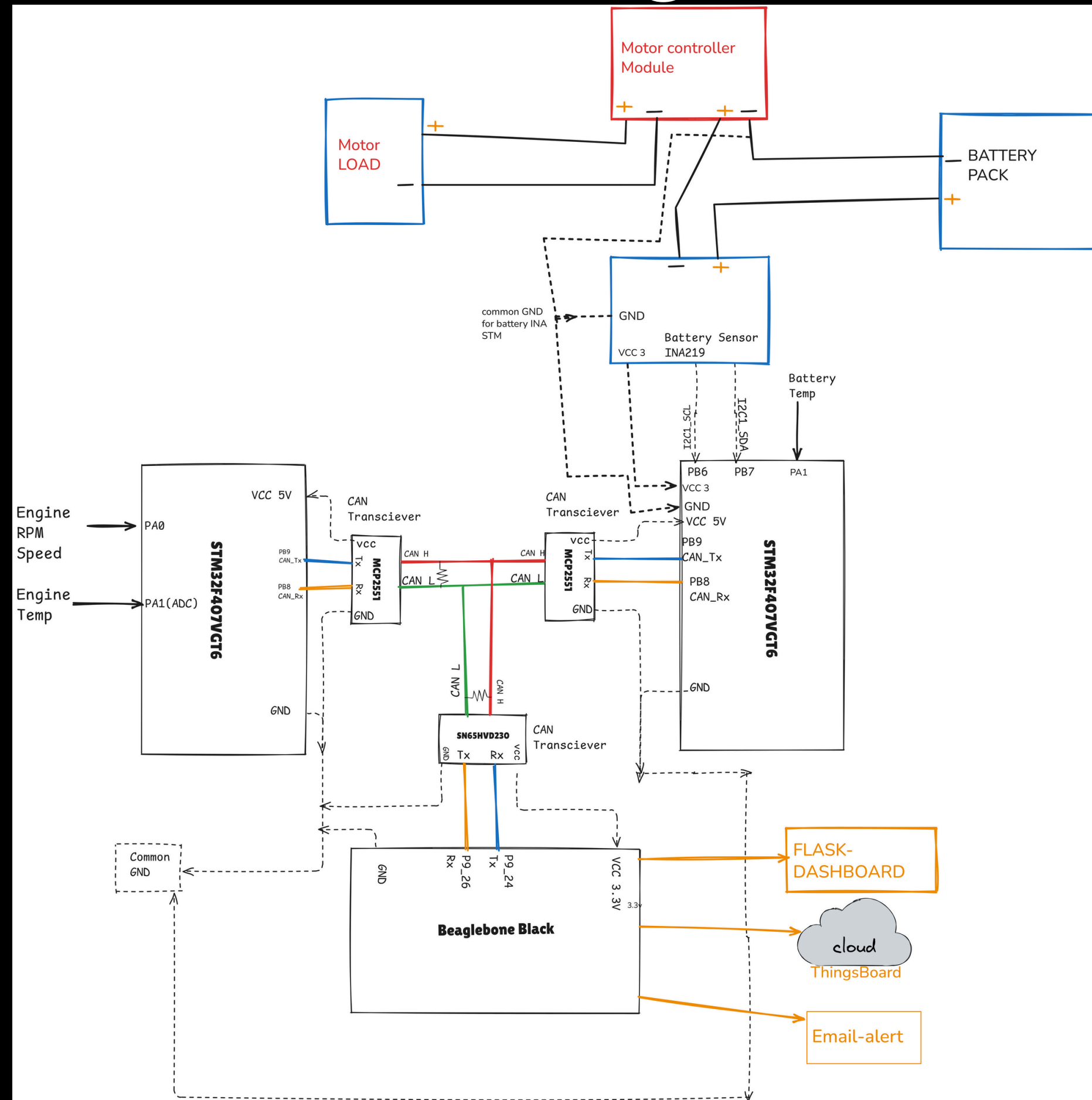
Sensors & Peripherals:

- Hall Effect
- LM35
- INA219 Current Sensor
- Battery Pack
- Motor



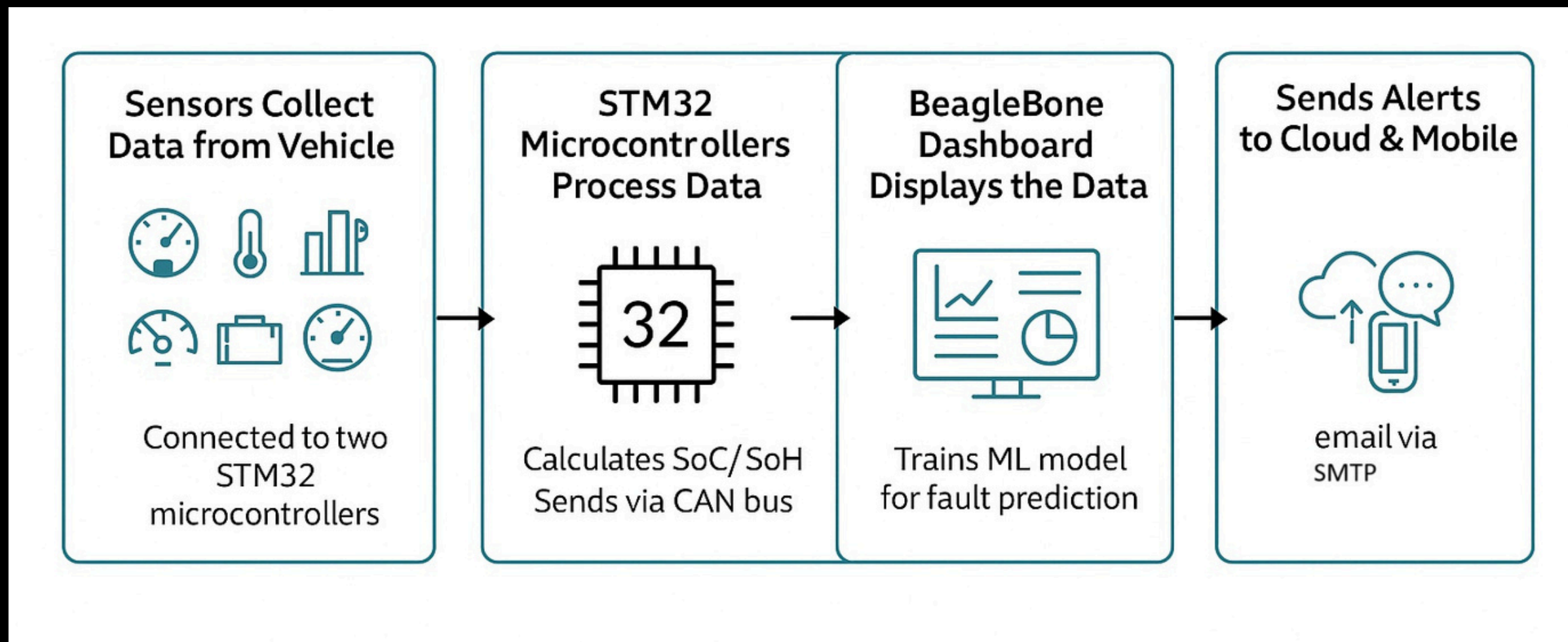
सी डैक
CDAC

अक्टस
acts



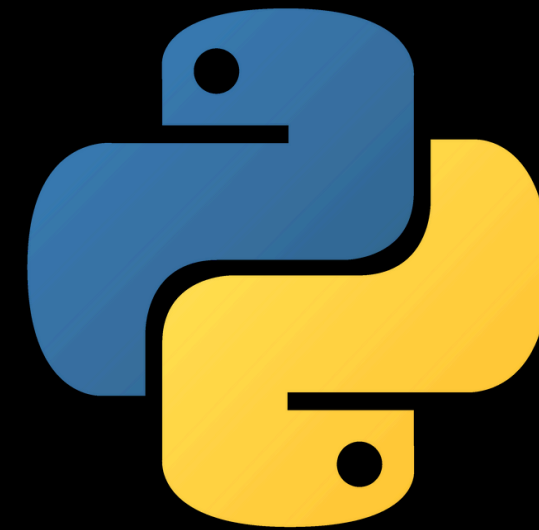
Data Flow & Integration

1. Sensors → STM32 → CAN bus → BeagleBone.
2. BeagleBone processes data → Flask dashboard + ThingsBoard (MQTT).
3. ML model detects faults → Email alerts.



Software Stack

- Firmware: STM32CubeIDE.
- Cloud: MQTT (ThingsBoard).
- Local UI: Flask + WebSocket.
- ML: scikit-learn (Decision Tree).



Flask



CAN Communication

- 500 kbps baud rate, MCP2551/SN65HVD230 transceivers.
- Message IDs for parameter differentiation (e.g., 0x123 for engine data & 0x124 for Battery data).
- Result: Zero message loss in 2+ hours of testing.

Predictive Maintenance (ML Model)

Model: Decision Tree (lightweight for edge deployment).

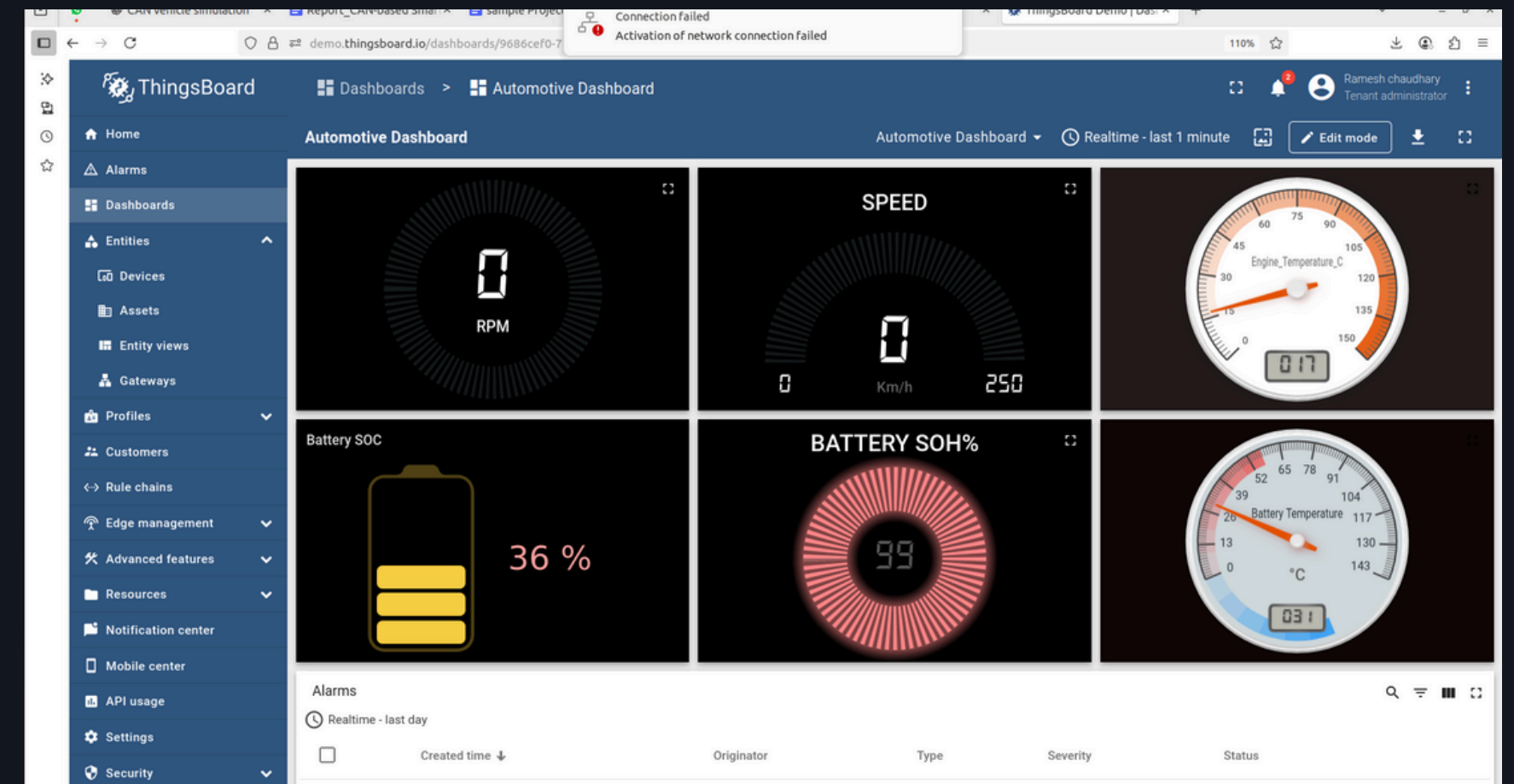
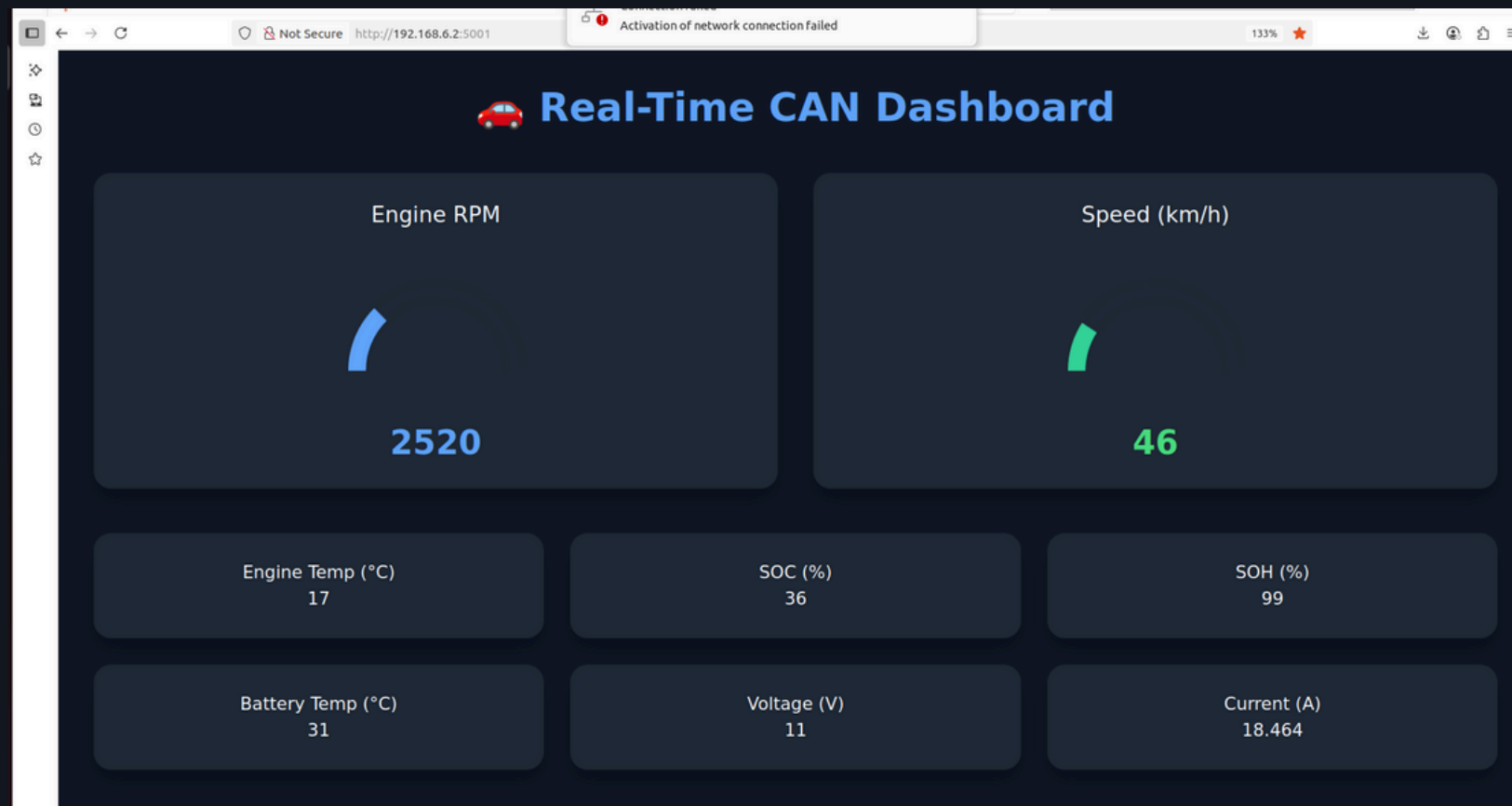
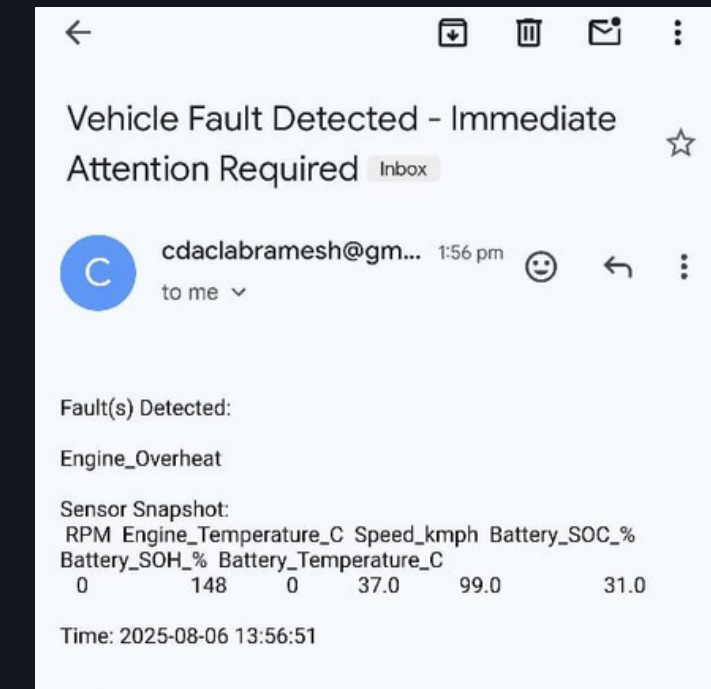
Features : RPM
Speed
Engine Temperatures
Battery metrics(SOC, SOH, Temperature)

Faults Detected : Engine Overheat
Overspeeding
High RPM
Battery Critical

Real-World Results

Dashboard:

Fault Detection: Engine overheat (100°C threshold) $\rightarrow$ Email alert in <10 sec.
High RPM >6000 etc.



Industry Applications

1. Automotive OEMs: Prototyping ECU diagnostics.
2. Fleet Management: Real-time telemetry.
3. EVs: Battery health monitoring.



Competitive Advantage

Why This System?

- Edge AI: Reduces cloud dependency (lower latency/cost).
- Modularity: Add nodes via CAN (e.g., ADAS, braking).
- Compliance: MQTT (QoS 1), ISO-compliant CAN.

Challenges :

- CAN bus arbitration (solved with prioritized message IDs).
- CAN bus synchronization with beaglebone.
- ML false positives (mitigated with threshold tuning).

Conclusion :

- Demonstrated a fully functional prototype with real-time monitoring, fault detection (96.1% accuracy), and seamless CAN-to-cloud integration.
- Demonstrated a cost-effective, scalable diagnostic system.

THANK YOU